Assignment report

gRPC Task Manager, Order API with MCP, Kong rate limiting,
and token bucket / leaky bucket algorithms.

Repository: https://github.com/Vinutha-P/task-manager.git
Language: Python 3. The same virtual environment builds and runs all three questions.

How to compile and run

From the repository root:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

cd question1
python -m grpc_tools.protoc -I proto --python_out=. --grpc_python_out=. proto/task.proto
python server.py          # terminal 1, listens on 127.0.0.1:50051
python client.py          # terminal 2

cd ../question3
python -m unittest test_rate_limiters.py
python demo.py

cd ../question2
python order_api.py       # terminal 1, http://127.0.0.1:5001
docker-compose up         # terminal 2, Kong on http://127.0.0.1:8000
ORDER_API_BASE=http://127.0.0.1:8000/api/orders python mcp_client_demo.py
python demo_rate_limit.py
```

Kong has no maintained native macOS package. question2/docker-compose.yml runs the official kong:3.9 image. Colima or Docker Desktop can provide the Docker daemon. question2/kong/kong.yml is the same gateway for a local kong binary, and question2/kong/register_via_admin_api.sh registers the service through the Admin API. The scripts directory also runs each question end to end: sh scripts/run_question1.sh, sh scripts/run_question3.sh, and sh scripts/run_question2.sh. Restart Kong between rate-limit runs. The limit is stored in memory and is 5 requests per minute per IP.

Question 1 — gRPC Task Manager

The proto file defines Task, CreateTaskRequest, GetTaskRequest, UpdateTaskRequest, and TaskService. ListTasksRequest and TaskUpdate are the extra messages required by the streaming RPCs. The server keeps tasks in a dictionary, assigns incremental ids from 1, and accepts only the statuses pending, in_progress, and done. id < 1 returns INVALID_ARGUMENT, an unknown id returns NOT_FOUND, and a bad status returns FAILED_PRECONDITION. Every GetTask call uses a 1-second deadline. The deadline demonstration sends metadata x-simulate-delay-seconds: 2, the server sleeps for 2 seconds, and the client prints DEADLINE_EXCEEDED and continues.

question1/proto/task.proto

```
1  syntax = "proto3";
2
3  package taskmanager;
4
5  // Task is the in-memory record stored by TaskService.
6  message Task {
7    int32 id = 1;
8    string title = 2;
9    // One of: "pending", "in_progress", "done".
10   string status = 3;
11 }
```

```
12
13  message CreateTaskRequest {
14    string title = 1;
15    // Optional. Defaults to "pending" when empty.
16    string status = 2;
17  }
18
19  message GetTaskRequest {
20    int32 id = 1;
21  }
22
23  message UpdateTaskRequest {
24    int32 id = 1;
25    string status = 2;
26  }
27
28  // Empty input for the ListTasks server stream.
29  message ListTasksRequest {}
30
31  // One status change sent by the client on the bidirectional stream.
32  message TaskUpdate {
33    int32 id = 1;
34    string status = 2;
35  }
36
37  service TaskService {
38    // 1. Create a new task.
39    rpc CreateTask(CreateTaskRequest) returns (Task);
40
41    // 2. Fetch a task by ID.
42    rpc GetTask(GetTaskRequest) returns (Task);
43
44    // 3. List all tasks (server streaming).
45    rpc ListTasks(ListTasksRequest) returns (stream Task);
46
47    // 4. Update task status.
48    rpc UpdateTask(UpdateTaskRequest) returns (Task);
49
50    // 5. Stream task updates (bidirectional streaming).
51    rpc StreamTaskUpdates(stream TaskUpdate) returns (stream Task);
52  }
```

question1/server.py — handlers and validation

```
48          self._require_valid_status(status, context)
49          with self._lock:
50              task_id = self._next_id
51              self._next_id += 1
52              record = TaskRecord(task_id, title, status)
53              self._tasks[task_id] = record
54                   print(f"CreateTask id={record.id} title={record.title!r} status={record.status}",
flush=True)
55              return record.to_proto()
56
57      def GetTask(self, request, context):
58          self._require_positive_id(request.id, context)
59          delay = _delay_seconds(context)
60          if delay > 0:
61              print(f"GetTask id={request.id} sleeping {delay:.1f}s", flush=True)
62              time.sleep(delay)
63              if not context.is_active():
64                  return task_pb2.Task()
65          record = self._require_task(request.id, context)
66          print(f"GetTask id={record.id} status={record.status}", flush=True)
67          return record.to_proto()
68
69      def ListTasks(self, request, context):
70          with self._lock:
71              records = [record.to_proto() for record in self._tasks.values()]
72          print(f"ListTasks streaming {len(records)} task(s)", flush=True)
73          for task in records:
74              if not context.is_active():
75                  return
76              yield task
```

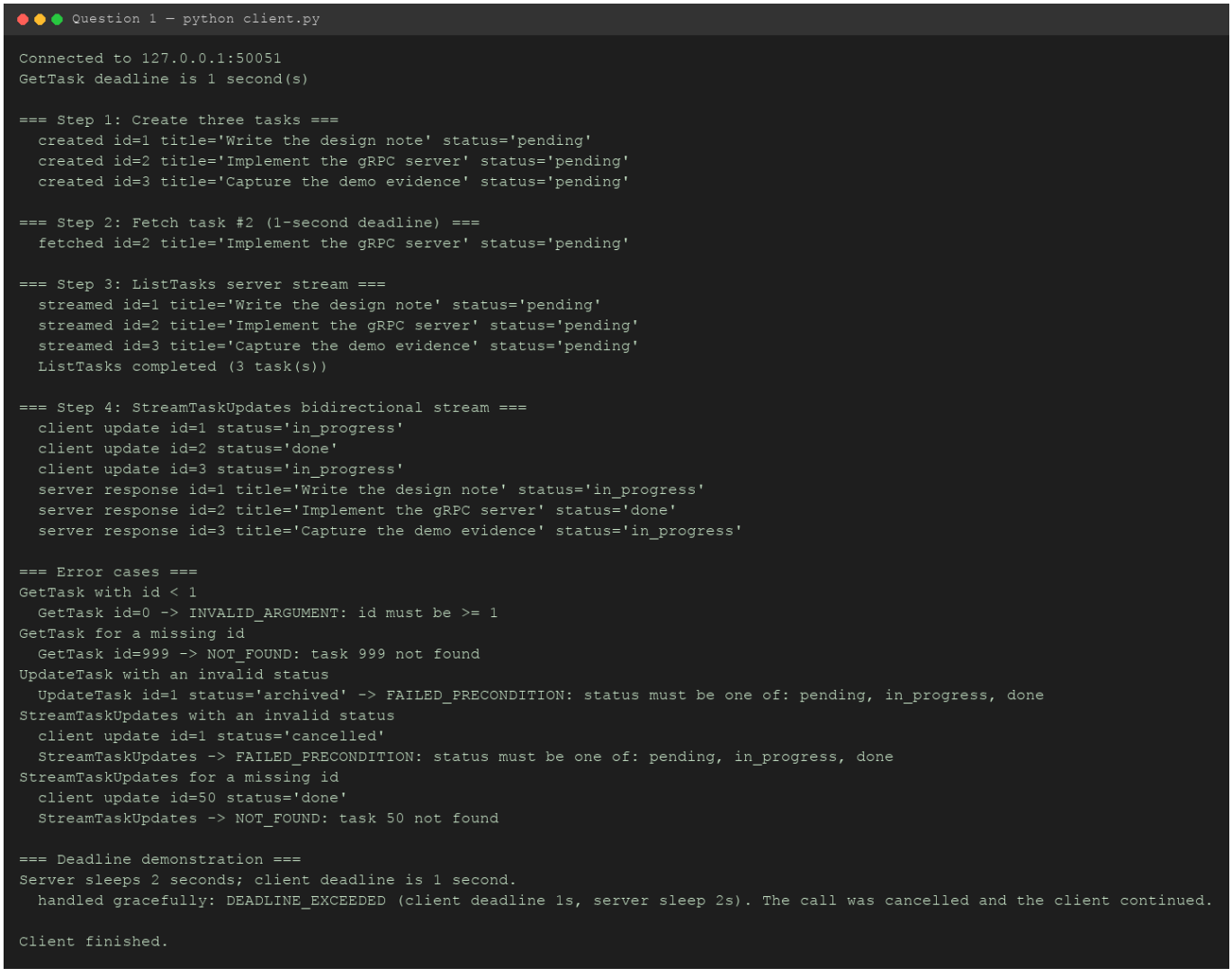
```
77
78      def UpdateTask(self, request, context):
79          updated = self._update(request.id, request.status, context)
80          print(f"UpdateTask id={updated.id} status={updated.status}", flush=True)
81          return updated
82
83      def StreamTaskUpdates(self, request_iterator, context):
84          for update in request_iterator:
85              if not context.is_active():
86                  return
87              updated = self._update(update.id, update.status, context)
88              print(
89                  f"StreamTaskUpdates id={updated.id} status={updated.status}",
90                  flush=True,
91              )
92              yield updated
93
94      def _update(self, task_id: int, status: str, context) -> task_pb2.Task:
95          self._require_positive_id(task_id, context)
96          self._require_valid_status(status, context)
97          record = self._require_task(task_id, context)
98          with self._lock:
99              current = self._tasks.get(task_id)
100             if current is None:
101                 context.abort(grpc.StatusCode.NOT_FOUND, f"task {task_id} not found")
102             current.status = status.strip()
103             return current.to_proto()
104
105     def _require_task(self, task_id: int, context) -> TaskRecord:
106         with self._lock:
107             record = self._tasks.get(task_id)
108         if record is None:
109             context.abort(grpc.StatusCode.NOT_FOUND, f"task {task_id} not found")
110         return record
111
112     @staticmethod
113     def _require_positive_id(task_id: int, context) -> None:
114         if task_id < 1:
115             context.abort(grpc.StatusCode.INVALID_ARGUMENT, "id must be >= 1")
116
117     @staticmethod
118     def _require_valid_status(status: str, context) -> None:
119         if status not in VALID_STATUSES:
120             context.abort(
121                 grpc.StatusCode.FAILED_PRECONDITION,
122                 "status must be one of: pending, in_progress, done",
123             )
124
125
126 def _delay_seconds(context) -> float:
127     for key, value in context.invocation_metadata():
128         if key.lower() == DELAY_METADATA_KEY:
129             try:
130                 return max(0.0, float(value))
131             except ValueError:
132                 context.abort(
133                     grpc.StatusCode.INVALID_ARGUMENT,
134                     "x-simulate-delay-seconds must be a number",
135                 )
136     return 0.0
137
```

question1/client.py — 1-second deadline and graceful timeout

```
40  def get_task(stub, task_id: int, delay_seconds: float = 0.0) -> None:
41      metadata = []
42      if delay_seconds > 0:
43          metadata.append(("x-simulate-delay-seconds", str(delay_seconds)))
44      try:
45          task = stub.GetTask(
46              task_pb2.GetTaskRequest(id=task_id),
47              timeout=DEADLINE_SECONDS,
48              metadata=metadata,
49          )
```

```
50      except grpc.RpcError as error:
51          if error.code() == grpc.StatusCode.DEADLINE_EXCEEDED:
52              print(
53                  "  handled gracefully: DEADLINE_EXCEEDED "
54                  f"(client deadline {DEADLINE_SECONDS:.0f}s, "
55                  f"server sleep {delay_seconds:.0f}s). "
56                  "The call was cancelled and the client continued."
57              )
58              return
59          print(f"  GetTask id={task_id} -> {_status_line(error)}")
60          return
61      print(f"  fetched id={task.id} title={task.title!r} status={task.status!r}")
62
```

Screenshot: client creates three tasks, fetches task 2, streams the list, applies three updates, and shows the three errors plus the timeout.



Screenshot: server log. GetTask for the deadline case sleeps 2.0 seconds.

```
Question 1 — python server.py

TaskService listening on 127.0.0.1:50051
CreateTask id=1 title='Write the design note' status=pending
CreateTask id=2 title='Implement the gRPC server' status=pending
CreateTask id=3 title='Capture the demo evidence' status=pending
GetTask id=2 status=pending
ListTasks streaming 3 task(s)
StreamTaskUpdates id=1 status=in_progress
StreamTaskUpdates id=2 status=done
StreamTaskUpdates id=3 status=in_progress
GetTask id=2 sleeping 2.0s
```

Question 2 — Order API, MCP tool, and Kong

Flow used in the demonstration: MCP client -> MCP server (tool order_api) -> Kong route /api/orders -> Order API. The API stores orders in memory. Each order has an integer id, item, quantity, and status created. quantity must be an integer greater than 0. Invalid input returns HTTP 400, a missing order returns HTTP 404, and a successful create returns HTTP 201. The MCP server validates arguments with the JSON Schema in question2/schemas/order_api_tool.json before it makes an HTTP call. create_order calls POST /orders, get_order calls GET /orders/{id}, and list_orders calls GET /orders. Kong registers service order-api, route /api/orders, and the rate-limiting plugin at 5 requests per minute. A pre-function plugin rewrites /api/orders to /orders. One nginx worker is used so the local counter is global.

question2/order_api.py — validation and endpoints

```
 44  def _parse_order_body():
 45      if not request.is_json:
 46          return None, _error("Content-Type must be application/json", 415)
 47      data = request.get_json(silent=True)
 48      if not isinstance(data, dict):
 49          return None, _error("request body must be a JSON object", 400)
 50      item = data.get("item")
 51      quantity = data.get("quantity")
 52      if not isinstance(item, str) or not item.strip():
 53          return None, _error("item is required and must be a non-empty string", 400)
 54      if isinstance(quantity, bool) or not isinstance(quantity, int):
 55          return None, _error("quantity must be an integer greater than 0", 400)
 56      if quantity <= 0:
 57          return None, _error("quantity must be an integer greater than 0", 400)
 58      return {"item": item.strip(), "quantity": quantity}, None
 59
 60
 61  def create_order():
 62      global _next_id
 63      payload, error = _parse_order_body()
 64      if error is not None:
 65          return error
 66      with _lock:
 67          order_id = _next_id
 68          _next_id += 1
 69          order = {
 70              "id": order_id,
 71              "item": payload["item"],
 72              "quantity": payload["quantity"],
 73              "status": "created",
 74          }
 75          _orders[order_id] = order
 76          print(f"POST order id={order['id']} item={order['item']!r} quantity={order['quantity']}",
flush=True)
 77      return jsonify(order), 201
 78
 79
 80  def get_order(order_id: int):
 81      with _lock:
```

```
        order = _orders.get(order_id)
    if order is None:
        return _error(f"order {order_id} not found", 404)
    return jsonify(order), 200


def list_orders():
    with _lock:
        orders = list(_orders.values())
    return jsonify(orders), 200

```

question2/mcp_server.py — schema check and routing

```
def validate_arguments(arguments: dict) -> None:
    errors = sorted(VALIDATOR.iter_errors(arguments), key=lambda item: list(item.path))
    if not errors:
        return
    messages = []
    for error in errors:
        location = ".".join(str(part) for part in error.path) or "arguments"
        messages.append(f"{location}: {error.message}")
    raise ValueError("; ".join(messages))


def call_order_api(arguments: dict, api_base: str | None = None) -> dict:
    """Validate arguments and return the upstream JSON response."""
    if not isinstance(arguments, dict):
        raise ValueError("arguments must be an object")
    validate_arguments(arguments)
    root = (api_base or base_url()).rstrip("/")
    action = arguments["action"]
    if action == "create_order":
        response = requests.post(
            root,
            json={"item": arguments["item"], "quantity": arguments["quantity"]},
            timeout=5,
        )
    elif action == "get_order":
        response = requests.get(f"{root}/{arguments['id']}", timeout=5)
    elif action == "list_orders":
        response = requests.get(root, timeout=5)
    else:
        raise ValueError(f"unsupported action: {action}")

    try:
        body = response.json()
    except ValueError:
        body = {"error": response.text or "upstream returned a non-JSON body"}
    return {"status_code": response.status_code, "body": body}

```

question2/kong/kong.docker.yml — service, route, and 5/minute limit

```
_format_version: "3.0"

# Same gateway config as kong.yml, with an upstream host that Docker can reach.

services:
  - name: order-api
    url: http://host.docker.internal:5001
    connect_timeout: 5000
    read_timeout: 5000
    write_timeout: 5000
    routes:
      - name: order-api-route
        paths:
          - /api/orders
        strip_path: false
        protocols:
          - http
    plugins:
      - name: rate-limiting
        config:
```

```
            minute: 5
            policy: local
            limit_by: ip
            fault_tolerant: true
            hide_client_headers: false
            error_code: 429
            error_message: "API rate limit exceeded (5 requests per minute)"
        - name: pre-function
          config:
            access:
              - |
                local path = kong.request.get_path()
                local new_path = path:gsub("^/api", "", 1)
                if new_path == "" then
                  new_path = "/"
                end
                kong.service.request.set_path(new_path)
```

question2/docker-compose.yml

```
services:
  kong:
    image: kong:3.9
    environment:
      KONG_DATABASE: "off"
      KONG_DECLARATIVE_CONFIG: /kong/kong.yml
      KONG_PROXY_LISTEN: 0.0.0.0:8000
      KONG_ADMIN_LISTEN: 0.0.0.0:8001
      KONG_UNTRUSTED_LUA: "on"
      KONG_NGINX_WORKER_PROCESSES: "1"
      KONG_LOG_LEVEL: notice
    volumes:
      - ./kong/kong.docker.yml:/kong/kong.yml:ro
    ports:
      - "8000:8000"
      - "8001:8001"
    extra_hosts:
      - "host.docker.internal:host-gateway"
    healthcheck:
      test: ["CMD", "kong", "health"]
      interval: 5s
      timeout: 5s
      retries: 20
```

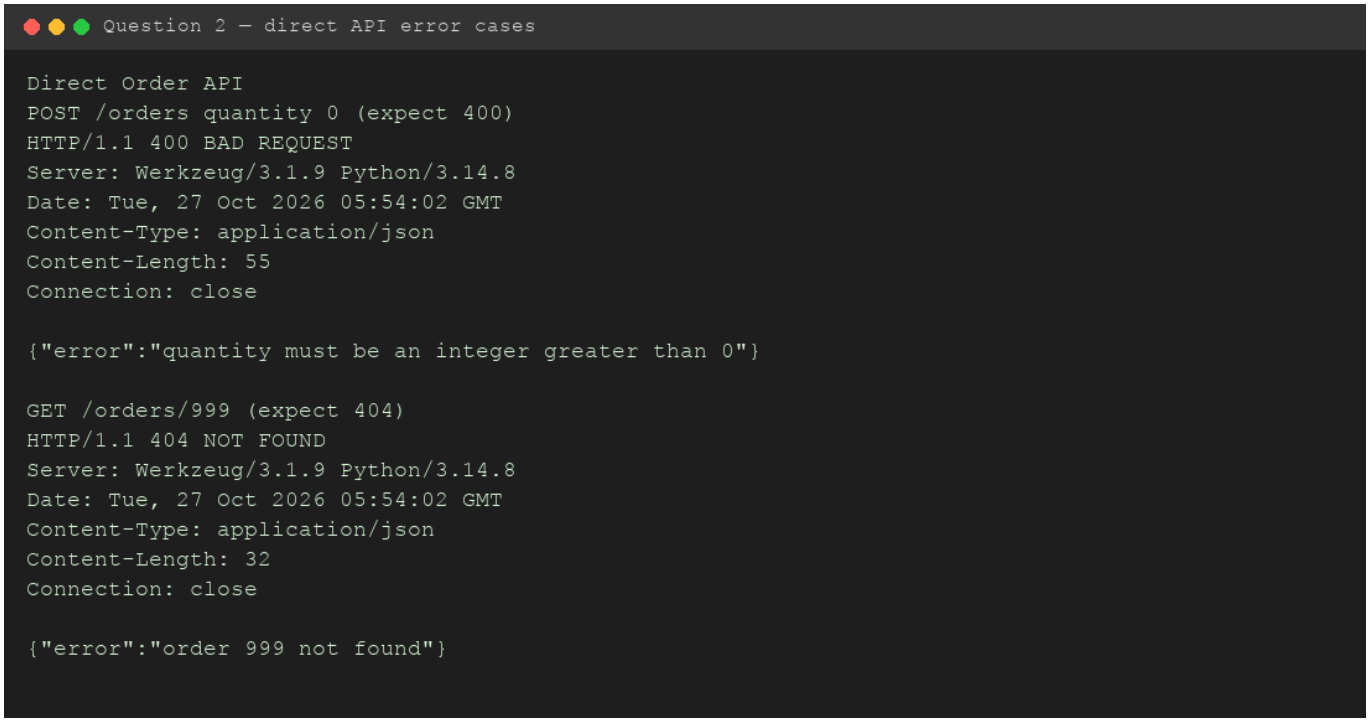
Screenshot: browser open on http://127.0.0.1:5001/orders, the Order API backend.

[{"id":1,"item":"pencil","quantity":3,"status":"created"},{"id":2,"item":"burst-1","quantity":1,"status":"created"},{"id":3,"item":"burst-2","quantity":1,"status":"created
{"id":4,"item":"burst-3","quantity":1,"status":"created"},{"id":5,"item":"burst-4","quantity":1,"status":"created"},{"id":6,"item":"burst-5","quantity":1,"status":"created
{"id":7,"item":"notebook","quantity":2,"status":"created"},{"id":8,"item":"pencil","quantity":5,"status":"created"}]

Screenshot: browser open on http://127.0.0.1:8000/api/orders, the same orders through the Kong route.

[{"id":1,"item":"pencil","quantity":3,"status":"created"},{"id":2,"item":"burst-1","quantity":1,"status":"created"},{"id":3,"item":"burst-2","quantity":1,"status":"created
{"id":4,"item":"burst-3","quantity":1,"status":"created"},{"id":5,"item":"burst-4","quantity":1,"status":"created"},{"id":6,"item":"burst-5","quantity":1,"status":"created
{"id":7,"item":"notebook","quantity":2,"status":"created"},{"id":8,"item":"pencil","quantity":5,"status":"created"}]

Screenshot: direct API rejects quantity 0 with HTTP 400 and an unknown id with HTTP 404.



Screenshot: MCP stdio session. The tool name is order_api. Creates, get, and list go through Kong. Quantity 0 is rejected by the schema, and a missing order returns the API 404 JSON.

Question 2 — MCP client through Kong

```
ORDER_API_BASE=http://127.0.0.1:8000/api/orders

--- initialize ---
{
  "protocolVersion": "2024-11-05",
  "capabilities": {
    "tools": {
      "listChanged": false
    }
  },
  "serverInfo": {
    "name": "order-api-mcp",
    "version": "1.0.0"
  }
}

--- tools/list (tool name order_api) ---
{
  "tools": [
    {
      "name": "order_api",
      "description": "Call the Order API. create_order posts a new order, get_order fetches one order by id, and list_orders returns every order. Requests go through the Kong route /api/orders.",
      "inputSchema": {
        "$schema": "https://json-schema.org/draft/2020-12/schema",
        "type": "object",
        "additionalProperties": false,
        "required": [
          "action"
        ],
        "properties": {
          "action": {
            "type": "string",
            "enum": [
              "create_order",
              "get_order",
              "list_orders"
            ],
            "description": "Which Order API operation to call."
          },
          "item": {
            "type": "string",
            "minLength": 1,
            "description": "Item name. Required when action is create_order."
          },
          "quantity": {
            "type": "integer",
            "minimum": 1,
            "description": "Quantity greater than 0. Required when action is create_order."
          },
          "id": {
            "type": "integer",
            "minimum": 1,
            "description": "Order id. Required when action is get_order."
          }
        },
        "allOf": [
          {
            "if": {
              "properties": {
                "action": {
                  "const": "create_order"
                }
              },
              "required": [
                "action"
              ]
            },
            "then": {
              "required": [
                "item",
                "quantity"
              ]
            }
          },
          {
            "if": {
              "properties": {
                "action": {
                  "const": "get_order"
                }
              },
              "required": [
                "action"
              ]
            },
            "then": {
              "required": [
                "id"
              ]
            }
          }
        ]
      }
    }
  ]
}

--- create_order notebook quantity 2 ---
isError=False
{
  "status_code": 201,
  "body": {
    "id": 7,
    "item": "notebook",
    "quantity": 2,
    "status": "created"
  }
}

--- create_order pencil quantity 5 ---
isError=False
{
  "status_code": 201,
  "body": {
```



Screenshot: seven POSTs to the Kong route. Requests 1-5 return HTTP 201 and the remaining-minute counter falls from 4 to 0. Requests 6 and 7 return HTTP 429 from kong/3.9.4.

Question 2 — Kong rate limit, 7 requests

```
Kong route: POST http://127.0.0.1:8000/api/orders
Plugin: rate-limiting minute=5 policy=local limit_by=ip
Sending 7 requests. Requests 6 and 7 must be HTTP 429.

===== request 1 =====
HTTP/1.1 201 CREATED
Content-Type: application/json
Content-Length: 58
Connection: keep-alive
X-RateLimit-Remaining-Minute: 4
RateLimit-Limit: 5
RateLimit-Remaining: 4
RateLimit-Reset: 58
X-RateLimit-Limit-Minute: 5
Server: Werkzeug/3.1.9 Python/3.14.8
Date: Tue, 27 Oct 2026 05:54:02 GMT
X-Kong-Upstream-Latency: 2
X-Kong-Proxy-Latency: 0
Via: 1.1 kong/3.9.4
X-Kong-Request-Id: b663862b52aa517935693913f6d2333a

{"id":2,"item":"burst-1","quantity":1,"status":"created"}


===== request 2 =====
HTTP/1.1 201 CREATED
Content-Type: application/json
Content-Length: 58
Connection: keep-alive
X-RateLimit-Remaining-Minute: 3
X-RateLimit-Limit-Minute: 5
RateLimit-Remaining: 3
RateLimit-Reset: 58
RateLimit-Limit: 5
Server: Werkzeug/3.1.9 Python/3.14.8
Date: Tue, 27 Oct 2026 05:54:02 GMT
X-Kong-Upstream-Latency: 3
X-Kong-Proxy-Latency: 0
Via: 1.1 kong/3.9.4
X-Kong-Request-Id: 9d1382489f35937f2056cb6884d3414b

{"id":3,"item":"burst-2","quantity":1,"status":"created"}


===== request 3 =====
HTTP/1.1 201 CREATED
Content-Type: application/json
Content-Length: 58
Connection: keep-alive
X-RateLimit-Remaining-Minute: 2
X-RateLimit-Limit-Minute: 5
RateLimit-Remaining: 2
RateLimit-Reset: 58
RateLimit-Limit: 5
Server: Werkzeug/3.1.9 Python/3.14.8
Date: Tue, 27 Oct 2026 05:54:02 GMT
```

```
X-Kong-Upstream-Latency: 1
X-Kong-Proxy-Latency: 1
Via: 1.1 kong/3.9.4
X-Kong-Request-Id: 24bb9d4dacba080b8b9db728cf34343f

{"id":4,"item":"burst-3","quantity":1,"status":"created"}


===== request 4 =====
HTTP/1.1 201 CREATED
Content-Type: application/json
Content-Length: 58
Connection: keep-alive
X-RateLimit-Remaining-Minute: 1
X-RateLimit-Limit-Minute: 5
RateLimit-Remaining: 1
RateLimit-Reset: 58
RateLimit-Limit: 5
Server: Werkzeug/3.1.9 Python/3.14.8
Date: Tue, 27 Oct 2026 05:54:02 GMT
X-Kong-Upstream-Latency: 2
X-Kong-Proxy-Latency: 0
Via: 1.1 kong/3.9.4
X-Kong-Request-Id: ef8d4c7710f6d227cf5c1c6e1b788e37

{"id":5,"item":"burst-4","quantity":1,"status":"created"}


===== request 5 =====
HTTP/1.1 201 CREATED
Content-Type: application/json
Content-Length: 58
Connection: keep-alive
X-RateLimit-Remaining-Minute: 0
X-RateLimit-Limit-Minute: 5
RateLimit-Remaining: 0
RateLimit-Reset: 58
RateLimit-Limit: 5
Server: Werkzeug/3.1.9 Python/3.14.8
Date: Tue, 27 Oct 2026 05:54:02 GMT
X-Kong-Upstream-Latency: 2
X-Kong-Proxy-Latency: 0
Via: 1.1 kong/3.9.4
X-Kong-Request-Id: 7a26fe3836448d489998b07bcb359f59

{"id":6,"item":"burst-5","quantity":1,"status":"created"}


===== request 6 =====
HTTP/1.1 429 Too Many Requests
Date: Tue, 27 Oct 2026 05:54:02 GMT
Content-Type: application/json; charset=utf-8
Connection: keep-alive
Retry-After: 58
X-RateLimit-Remaining-Minute: 0
X-RateLimit-Limit-Minute: 5
RateLimit-Remaining: 0
RateLimit-Reset: 58
RateLimit-Limit: 5
```

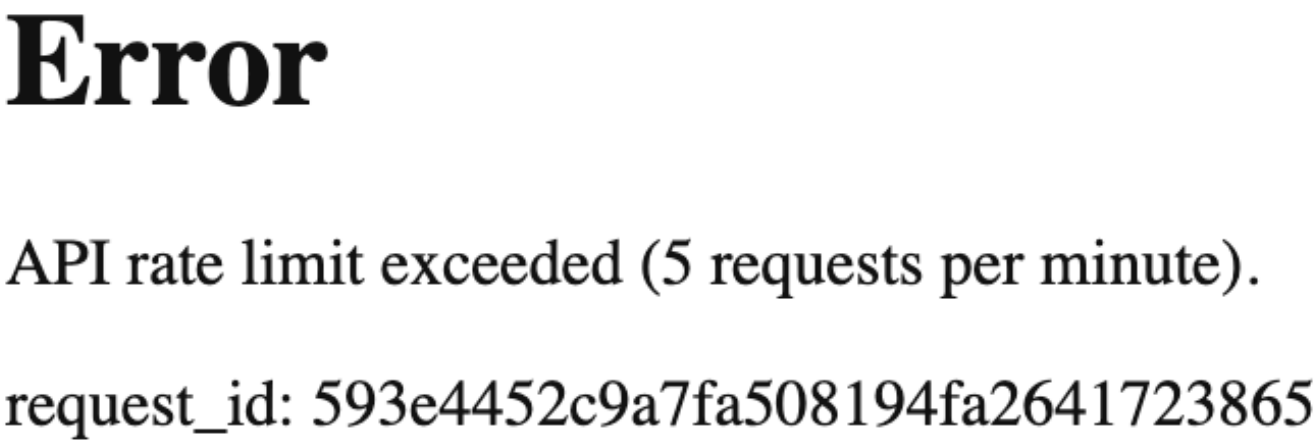
```
Content-Length: 116
X-Kong-Response-Latency: 1
Server: kong/3.9.4
X-Kong-Request-Id: ec20234fe66f0823e00c68728da80540

{
  "message":"API rate limit exceeded (5 requests per minute)",
  "request_id":"ec20234fe66f0823e00c68728da80540"
}

===== request 7 =====
HTTP/1.1 429 Too Many Requests
Date: Tue, 27 Oct 2026 05:54:02 GMT
Content-Type: application/json; charset=utf-8
Connection: keep-alive
Retry-After: 58
X-RateLimit-Remaining-Minute: 0
X-RateLimit-Limit-Minute: 5
RateLimit-Remaining: 0
RateLimit-Reset: 58
RateLimit-Limit: 5
Content-Length: 116
X-Kong-Response-Latency: 1
Server: kong/3.9.4
X-Kong-Request-Id: 55d47a175887e62399218d6e4571352a

{
  "message":"API rate limit exceeded (5 requests per minute)",
  "request_id":"55d47a175887e62399218d6e4571352a"
}
```

Screenshot: browser on http://127.0.0.1:8000/api/orders after the limit is exhausted. Kong responds: API rate limit exceeded (5 requests per minute).



Screenshot: unit tests for the Order API and the MCP tool.

```
Question 2 — API and MCP unit tests

POST order id=1 item='notebook' quantity=2
.POST order id=1 item='bag' quantity=1
.POST order id=1 item='pen' quantity=1
POST order id=2 item='pencil' quantity=3
.....127.0.0.1 - - [27/Oct/2026 11:21:49] "GET /health HTTP/1.1" 200 -
POST order id=1 item='stapler' quantity=1
127.0.0.1 - - [27/Oct/2026 11:21:49] "POST /orders HTTP/1.1" 201 -
127.0.0.1 - - [27/Oct/2026 11:21:49] "GET /orders/1 HTTP/1.1" 200 -
127.0.0.1 - - [27/Oct/2026 11:21:49] "GET /orders HTTP/1.1" 200 -
.127.0.0.1 - - [27/Oct/2026 11:21:49] "GET /orders/999 HTTP/1.1" 404 -
...
----------------------------------------------------------------------
Ran 11 tests in 0.044s

OK
```

Question 3 — Token bucket and leaky bucket

TokenBucket starts full. It refills continuously at refill_rate tokens per second, capped at capacity. A request is allowed only when a whole token is available, so the caller can burst up to capacity and is then limited to the refill rate. LeakyBucket starts empty. Each allowed request adds one unit and the level leaks continuously at leak_rate units per second. A request that would raise the level above capacity is rejected, which smooths traffic to the leak rate. Both classes take an injectable clock so the tests advance time without sleeping. The demo prints a virtual-clock timeline and a short real-time burst.

question3/rate_limiters.py

```
"""Token bucket and leaky bucket rate limiters.

Token bucket
  The bucket holds tokens, up to ``capacity``, and refills continuously at
  ``refill_rate`` tokens per second. A request is allowed when at least one
  token is available. The bucket starts full, so a client may burst up to
  ``capacity`` immediately.

Leaky bucket
  The bucket holds the water level of recent requests, up to ``capacity``,
  and leaks continuously at ``leak_rate`` units per second. Each allowed
  request adds one unit. A request is rejected when adding it would overflow
  the bucket. The bucket starts empty. Output is smoothed to the leak rate;
  a burst larger than ``capacity`` is rejected.
"""

from __future__ import annotations

import threading
import time
from collections.abc import Callable


Clock = Callable[[], float]


class TokenBucket:
    def __init__(self, capacity: float, refill_rate: float, clock: Clock = time.monotonic) -> None:
        if capacity <= 0:
            raise ValueError("capacity must be > 0")
        if refill_rate <= 0:
            raise ValueError("refill_rate must be > 0")
        self.capacity = float(capacity)
        self.refill_rate = float(refill_rate)
        self._clock = clock
        self._tokens = float(capacity)
        self._last = clock()
```

```
        self._lock = threading.Lock()

    @property
    def tokens(self) -> float:
        with self._lock:
            self._refill(self._clock())
            return self._tokens

    def allow(self, amount: float = 1.0) -> bool:
        if amount <= 0:
            raise ValueError("amount must be > 0")
        with self._lock:
            now = self._clock()
            self._refill(now)
            if self._tokens < amount:
                return False
            self._tokens -= amount
            return True

    def _refill(self, now: float) -> None:
        elapsed = max(0.0, now - self._last)
        self._tokens = min(self.capacity, self._tokens + elapsed * self.refill_rate)
        self._last = now


class LeakyBucket:
    def __init__(self, capacity: float, leak_rate: float, clock: Clock = time.monotonic) -> None:
        if capacity <= 0:
            raise ValueError("capacity must be > 0")
        if leak_rate <= 0:
            raise ValueError("leak_rate must be > 0")
        self.capacity = float(capacity)
        self.leak_rate = float(leak_rate)
        self._clock = clock
        self._level = 0.0
        self._last = clock()
        self._lock = threading.Lock()

    @property
    def level(self) -> float:
        with self._lock:
            self._leak(self._clock())
            return self._level

    def allow(self, amount: float = 1.0) -> bool:
        if amount <= 0:
            raise ValueError("amount must be > 0")
        with self._lock:
            now = self._clock()
            self._leak(now)
            if self._level + amount > self.capacity:
                return False
            self._level += amount
            return True

    def _leak(self, now: float) -> None:
        elapsed = max(0.0, now - self._last)
        self._level = max(0.0, self._level - elapsed * self.leak_rate)
        self._last = now
```

Screenshot: token bucket allows a burst of 5 then denies, and leaky bucket allows a burst of 4 then denies. Later timestamps show refill and leak.

```
Question 3 — python demo.py

=== Token bucket (virtual clock) ===
capacity=5  refill_rate=1 token/second  starts full
t=0.0s  7 requests: ALLOW ALLOW ALLOW ALLOW ALLOW DENY DENY
t=2.0s  3 requests: ALLOW ALLOW DENY  (2 tokens refilled)
t=12.0s  6 requests: ALLOW ALLOW ALLOW ALLOW ALLOW DENY  (capped at 5 tokens)

=== Leaky bucket (virtual clock) ===
capacity=4  leak_rate=1 unit/second  starts empty
t=0.0s  6 requests: ALLOW ALLOW ALLOW ALLOW DENY DENY
t=2.0s  3 requests: ALLOW ALLOW DENY  (2 units leaked)
t=12.0s  5 requests: ALLOW ALLOW ALLOW ALLOW DENY  (bucket drained, then filled)

=== Real-time token bucket ===
capacity=3  refill_rate=10 tokens/second
immediate burst of 5: ALLOW ALLOW ALLOW DENY DENY
after 0.25s, burst of 3: ALLOW ALLOW DENY

=== Real-time leaky bucket ===
capacity=3  leak_rate=10 units/second
immediate burst of 5: ALLOW ALLOW ALLOW DENY DENY
after 0.25s, burst of 3: ALLOW ALLOW DENY

Demo finished.
```

Screenshot: eight unit tests passed.

```
Question 3 — unittest

........
----------------------------------------------------------------------
Ran 8 tests in 0.000s

OK
```

Requirement checklist

Question 1: CreateTask, GetTask, ListTasks server stream, UpdateTask, and StreamTaskUpdates bidirectional stream are implemented. Ids are incremental. Status is restricted to pending, in_progress, and done. The client creates three tasks, fetches task 2, lists every task, and prints three streamed updates. INVALID_ARGUMENT, NOT_FOUND, FAILED_PRECONDITION, and DEADLINE_EXCEEDED are all visible in the client screenshot.

Question 2: REST endpoints, in-memory orders, quantity validation, and JSON error codes are implemented. The MCP tool is named order_api and exposes create_order, get_order, and list_orders. Kong publishes route /api/orders with rate limiting set to 5 requests per minute. The terminal screenshot and the browser screenshot both show HTTP 429 after the fifth request.

Question 3: TokenBucket and LeakyBucket are implemented in Python, with tests and a runnable demo.